SOFTWARE REQUIREMENTS SPECIFICATION

Student Performance Prediction System

Machine Learning Regression Project

Document Type	Software Requirements Specification (SRS)
Version	1.0
Date	12 October 2026
Prepared For	Project documentation and academic review

Prepared from the supplied Jupyter Notebook.

Document Control

Version	Date	Description
1.0	12 October 2026	Initial SRS based on notebook review

Table of Contents

1. Introduction

2. Overall Description

3. System Requirements

4. Data Requirements

5. Model and Workflow

6. Constraints and Limitations

7. Testing and Acceptance

8. Future Enhancements

9. References

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements and expected behavior of the Student Performance Prediction project implemented in the supplied Jupyter Notebook. It documents the project's scope, inputs, processing, model, outputs, constraints, and test criteria.

1.2 Scope

The system uses student-related factors to predict the target variable Exam_Score using a machine-learning regression pipeline. It loads a CSV dataset, separates predictors and target, preprocesses numerical and categorical features, trains a regression model, evaluates its predictions, and plots actual versus predicted exam scores.

1.3 Intended Audience

This document is intended for the project owner, students, instructors, reviewers, and developers who need to understand, run, assess, or extend the notebook.

1.4 Definitions and Acronyms

SRS: Software Requirements Specification. Feature: an input variable used by the model. Target: the value to be predicted (Exam_Score). Imputation: replacing missing values. One-hot encoding: representing categories as binary columns. R^2: coefficient of determination. MAE: mean absolute error. RMSE: root mean squared error.

2. Overall Description

2.1 Product Perspective

The submitted artifact is a Python Jupyter Notebook, not a standalone deployed application. It depends on a CSV dataset and standard Python data-science and scikit-learn libraries.

2.2 Product Functions

The notebook imports required libraries; loads StudentPerformanceFactors.csv; separates features and target; identifies numerical and categorical columns; imputes missing values; scales numerical features and generates degree-2 polynomial features; encodes categorical features; splits data into training and test sets; fits LinearRegression; computes evaluation metrics; and plots actual versus predicted test scores.

2.3 User Classes

The primary users are students, instructors, or analysts who can run a Python notebook. The supplied code does not implement separate user accounts, authentication, or administrative roles.

2.4 Operating Environment

The project requires Python and Jupyter Notebook or a compatible notebook environment. The notebook imports pandas, NumPy, Matplotlib, Seaborn, and scikit-learn. The configured dataset path must be available in the execution environment.

2.5 Assumptions and Dependencies

The CSV file must be accessible and contain a column named Exam_Score. The notebook identifies int64 and float64 columns as numerical and object columns as categorical. Compatible versions of the imported libraries must be installed.

3. System Requirements

3.1 Functional Requirements

FR-01: The system shall load StudentPerformanceFactors.csv from the configured path.\nFR-02: The system shall display the dataset shape and a preview of its first rows.\nFR-03: The system shall use all columns except Exam_Score as input features and Exam_Score as the target.\nFR-04: The system shall identify numerical and object-type categorical input columns.\nFR-05: The system shall impute missing numerical values using the median and missing categorical values using the most frequent value.\nFR-06: The system shall standardize numerical features and generate degree-2 polynomial features.\nFR-07: The system shall one-hot encode categorical features and ignore unseen categories during transformation.\nFR-08: The system shall split data into 80% training and 20% test sets using random_state=42.\nFR-09: The system shall train a LinearRegression estimator through the preprocessing pipeline.\nFR-10: The system shall generate predictions for training and test records.\nFR-11: The system shall report training R^2, test R^2, test MAE, and test RMSE.\nFR-12: The system shall plot actual test scores against predicted scores and display a diagonal reference line.

3.2 Non-Functional Requirements

NFR-01 Usability: The workflow shall be organized into notebook cells that can be executed in sequence.\nNFR-02 Reproducibility: The train-test split shall use a fixed random_state value of 42.\nNFR-03 Maintainability: Data loading, preprocessing, model training, evaluation, and visualization shall remain logically separated.\nNFR-04 Robustness: The categorical encoder shall tolerate categories not seen during training.\nNFR-05 Data handling: Imputation shall occur within the preprocessing pipeline fitted on the training data.\nNFR-06 Performance: Runtime depends on dataset size, feature count, and available computing resources; no explicit runtime target is defined.\nNFR-07 Security and privacy: The notebook does not implement access control, encryption, or a data-retention policy. Student data should only be shared with appropriate authorization.

3.3 External Interface Requirements

User interface: notebook cells, printed outputs, and a Matplotlib/Seaborn chart.\nData interface: a CSV file with a header row and an Exam_Score column.\nSoftware interfaces: pandas, NumPy, Matplotlib, Seaborn, and scikit-learn APIs. The notebook does not implement a web API, database, mobile interface, or desktop GUI.

4. Data Requirements

4.1 Input Data

The input file is StudentPerformanceFactors.csv. It is expected to contain a tabular dataset with Exam_Score as the target and remaining columns as predictors. The notebook does not include a complete data dictionary, column definitions, units, or valid ranges; these should be confirmed from the dataset documentation.

4.2 Data Processing

Numerical features use median imputation, standard scaling, and degree-2 PolynomialFeatures with include_bias=False. Categorical features use most-frequent imputation followed by OneHotEncoder(handle_unknown='ignore'). A ColumnTransformer combines the transformations, and a Pipeline connects preprocessing to LinearRegression.

4.3 Output Data

The notebook generates predicted scores for training and test records, reports training R^2, test R^2, test MAE, and test RMSE, and displays a scatter plot of actual versus predicted test scores. It does not save predictions or the trained model to a file.

4.4 Data Quality

The pipeline handles missing values in supported numerical and object columns. It does not explicitly validate duplicate records, outliers, target-score ranges, unexpected data types, or all possible invalid values.

5. Model and Workflow

5.1 Model Design

The model is a scikit-learn Pipeline containing a ColumnTransformer preprocessor and a LinearRegression estimator. Numerical inputs are expanded into degree-2 polynomial terms, while categorical inputs are one-hot encoded. The notebook refers to this approach as Polynomial Regression.

5.2 Processing Workflow

1) Import dependencies. 2) Read the CSV dataset. 3) Separate features X and target y. 4) Identify numerical and categorical columns. 5) Configure preprocessing transformers. 6) Split the dataset into training and test sets. 7) Fit the pipeline on training data. 8) Predict training and test scores. 9) Calculate evaluation metrics. 10) Display the actual-versus-predicted plot.

5.3 Evaluation Criteria

R^2 measures how much target variation is explained by the predictions. Values closer to 1 generally indicate a stronger fit; a negative test R^2 can indicate performance worse than a constant-mean baseline. MAE is the average absolute prediction error. RMSE gives greater weight to larger errors. The notebook does not define numerical acceptance thresholds.

5.4 Model Quality Note

Polynomial features do not guarantee reduced overfitting. The model should be assessed using training-versus-test performance and, ideally, cross-validation and comparison with a simple baseline.

6. Constraints and Limitations

6.1 Constraints

The configured dataset path is environment-specific. The target column must be named exactly Exam_Score. The notebook explicitly detects int64/float64 numerical columns and object categorical columns; other data types may require additional handling.

6.2 Known Limitations

The notebook is an offline experiment rather than a deployed prediction service. It has no form for entering a new student's details, model persistence, comprehensive input validation, logging, authentication, or production interface. Model accuracy cannot be stated without executing the notebook on the intended dataset.

6.3 Out of Scope

A web or mobile application, database integration, automated data collection, student login, report export, and production deployment are not implemented in the supplied notebook.

7. Testing and Acceptance

7.1 Test Cases

TC-01 Dataset loading: verify that the CSV loads and the shape and preview are displayed.\nTC-02 Target validation: verify that Exam_Score exists; if absent, execution should stop with a clear error.\nTC-03 Column detection: verify that numerical and categorical feature lists are identified.\nTC-04 Missing values: verify that preprocessing completes when supported input columns contain missing values.\nTC-05 Data split: verify that training and test sets follow an 80/20 ratio, subject to rounding.\nTC-06 Model training: verify that model_pipeline.fit completes successfully.\nTC-07 Predictions: verify that training and test predictions are produced and match the lengths of their target arrays.\nTC-08 Metrics: verify that all four metric values are reported as numbers.\nTC-09 Visualization: verify that the scatter plot, labels, title, grid, and reference line are displayed.\nTC-10 Unseen categories: verify that prediction does not fail solely because a categorical value was not present in the training set.

7.2 Acceptance Criteria

The notebook is accepted for its stated scope when its cells execute successfully in a compatible environment with a valid dataset, the pipeline trains, predictions and all four metrics are produced, and the comparison plot is displayed. The supplied code does not define a numerical accuracy threshold.

7.3 Verification Status

This document is based on static review of the supplied notebook. Runtime execution and evaluation metric values have not been independently verified while preparing this SRS.

8. Future Enhancements

8.1 Recommendations

Validate required columns and target values; add a dataset data dictionary; compare the model against a baseline; use cross-validation and compare alternative models; inspect residuals; save the trained pipeline; provide a function or interface for predicting new records; export evaluation results and plots; and document dataset licensing and privacy requirements.

9. References

9.1 Project Source

Source reviewed: pr-project - Copy.ipynb, supplied by the project owner. The notebook references a Kaggle input path ending in StudentPerformanceFactors.csv.

9.2 Technical Libraries

pandas; NumPy; Matplotlib; Seaborn; scikit-learn, including Pipeline, ColumnTransformer, SimpleImputer, StandardScaler, PolynomialFeatures, OneHotEncoder, LinearRegression, train_test_split, and regression metrics. Library versions are not pinned in the notebook.